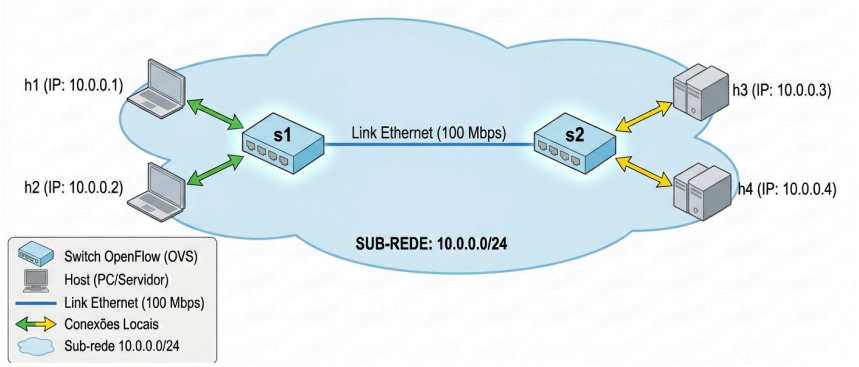
Relatório
Projeto prático - Rede SDN

Caio Ferreira - cfgs e Gabriel Monteiro Silva - gms2

Projeto prático - Rede SDN

Fase 1: A Infraestrutura Física (Topologia de Rede)

Nessa fase foi criado o script “topologia_mininet.py”, que utiliza o Mininet para criar uma rede com a topologia indicada.



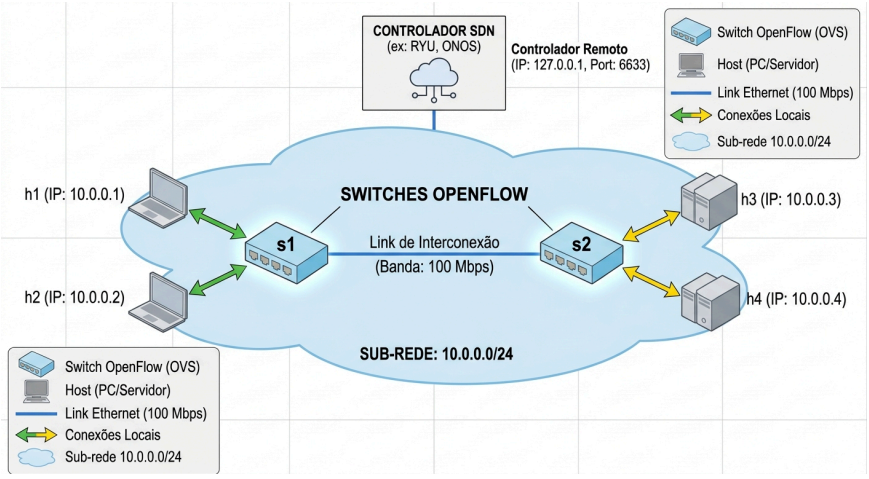
Ao iniciar a rede no mininet e executar o comando “pingall” o seguinte resultado foi obtido:

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X
h2 -> X X X
h3 -> X X X
h4 -> X X X
*** Results: 100% dropped (0/12 received)
```

Todos os pings foram dropados, o que é o resultado esperado já que os switches não têm regras de encaminhamento instaladas e não há controlador conectado para instruí-los.

Fase 2: O Cérebro da Rede (Conexão com o Controlador)

Assim como foi indicado, o script da rede mininet foi modificado para apontar para um controlador remoto e na hora de rodar o ryu foi executado em um terminal separado com um controlador vazio.



- Handshake OpenFlow:

A partir dos logs do controlador é possível observar que os switches s1 e s2 se conectaram a ele com sucesso.

```
connected socket:<eventlet.greenio.base.GreenSocket object at 0x78980ee02100> address:('127.0.0.1', 53628)
connected socket:<eventlet.greenio.base.GreenSocket object at 0x78980edf4cd0> address:('127.0.0.1', 53634)
hello ev <ryu.controller.ofp_event.EventOFPHello object at 0x78980eddd0a0>
move onto config mode
switch features ev version=0x4,msg_type=0x6,msg_len=0x20,xid=0x61660c94,OFPSwitchFeatures(auxiliary_id=0,capabilities=79
,datapath_id=1,n_buffers=0,n_tables=254)
move onto main mode
connected socket:<eventlet.greenio.base.GreenSocket object at 0x78980edf4b50> address:('127.0.0.1', 53640)
hello ev <ryu.controller.ofp_event.EventOFPHello object at 0x78980ee0e310>
move onto config mode
switch features ev version=0x4,msg_type=0x6,msg_len=0x20,xid=0xbab759a8,OFPSwitchFeatures(auxiliary_id=0,capabilities=79
,datapath_id=2,n_buffers=0,n_tables=254)
move onto main mode
```

- “connected socket: ... address” switches estabelecem a conexão TCP com o ryu, depois eles começam o Handshake OpenFlow:
- “hello ev <EventOFPHello>” é a mensagem de Hello que inicia a negociação de informações.
- “switch features ev <OFPSwitchFeatures> ... datapath_id=n” confirma a troca de informações e identifica os switches, onde n é o id do switch, n do s1 = 1, n do s2 = 2.
- “move onto config mode” conclui o handshake, switches entraram em modo operacional (“main mode”).

- Dump:

Ao executar o dump no mininet é possível ver todos os dispositivos que estão configurados na rede.

```
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=10570>
<Host h2: h2-eth0:10.0.0.2 pid=10572>
<Host h3: h3-eth0:10.0.0.3 pid=10574>
<Host h4: h4-eth0:10.0.0.4 pid=10576>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None,s1-eth3:None pid=10581>
<OVSSwitch s2: lo:127.0.0.1,s2-eth1:None,s2-eth2:None,s2-eth3:None pid=10584>
<RemoteController c0: 127.0.0.1:6653 pid=10564>
```

- Pingall:

No mininet foi executado o comando pingall novamente, mas para conseguir ver as mensagens Packet-In enviadas para o controlador foi necessário executar o controlador ryu com o script de debug “debug_controller.py” e executar os seguintes comandos no mininet antes de rodar o pingall novamente:

```
mininet> sh ovs-ofctl -O OpenFlow13 add-flow s1 "priority=0 actions=CONTROLLER"
mininet> sh ovs-ofctl -O OpenFlow13 add-flow s2 "priority=0 actions=CONTROLLER"
```

Após rodar o pingall, no mininet se obteve o mesmo resultado, mas no log do controlador ryu foram obtidas mensagens de packet-in indicando que a mensagem foi enviada para o controlador, que não sabe o que responder pois ainda está vazio.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X
h2 -> X X X
h3 -> X X X
h4 -> X X X
*** Results: 100% dropped (0/12 received)
```

```
EVENT ofp_event->DebugController EventOFPPacketIn
  Packet-In recebido do switch: 1
EVENT ofp_event->DebugController EventOFPPacketIn
  Packet-In recebido do switch: 2
```

Fase 3: Inteligência Básica (Encaminhamento L2 / MAC Learning)

Dessa vez o controlador ryu não é mais vazio, foi executado com o script simple_switch_13.py, que ao receber pacotes desconhecidos guarda a porta de origem e o endereço MAC e cria uma regra nos switches para que os próximos pacotes já sejam enviados diretamente, sem passar pelo controlador.

- Pingall:

Dessa vez ao executar o pingall nenhum pacote é perdido, pois o controlador agora trata os pacotes recebidos.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
```

- Regras:

Os prints abaixo mostram as regras que estão nos switchs s1 e s2, que foram criadas durante a execução do pingall.

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=96.450s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth3",dl_src=76:19:35:77:30:b7,dl_dst=22:16:23:8b:9a:25 actions=output:"s1-eth2"
 cookie=0x0, duration=96.459s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth2",dl_src=22:16:23:8b:9a:25,dl_dst=76:19:35:77:30:b7 actions=output:"s1-eth3"
 cookie=0x0, duration=96.446s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth1",dl_src=1a:f7:90:f3:d3:db,dl_dst=22:16:23:8b:9a:25 actions=output:"s1-eth2"
 cookie=0x0, duration=96.444s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth2",dl_src=22:16:23:8b:9a:25,dl_dst=1a:f7:90:f3:d3:db actions=output:"s1-eth1"
 cookie=0x0, duration=96.429s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth1",dl_src=72:7d:a2:5f:00:d0,dl_dst=22:16:23:8b:9a:25 actions=output:"s1-eth2"
 cookie=0x0, duration=96.427s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth2",dl_src=22:16:23:8b:9a:25,dl_dst=72:7d:a2:5f:00:d0 actions=output:"s1-eth1"
 cookie=0x0, duration=96.403s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth1",dl_src=1a:f7:90:f3:d3:db,dl_dst=76:19:35:77:30:b7 actions=output:"s1-eth3"
 cookie=0x0, duration=96.400s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth3",dl_src=76:19:35:77:30:b7,dl_dst=1a:f7:90:f3:d3:db actions=output:"s1-eth1"
 cookie=0x0, duration=96.381s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth1",dl_src=72:7d:a2:5f:00:d0,dl_dst=76:19:35:77:30:b7 actions=output:"s1-eth3"
 cookie=0x0, duration=96.378s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth3",dl_src=76:19:35:77:30:b7,dl_dst=72:7d:a2:5f:00:d0 actions=output:"s1-eth1"
 cookie=0x0, duration=120.331s, table=0, n_packets=60, n_bytes=4360, priority=0 actions=CONTROLLER:65535
```

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2
 cookie=0x0, duration=345.736s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s2-eth2",dl_src=1a:f7:90:f3:d3:db,dl_dst=22:16:23:8b:9a:25 actions=output:"s2-eth1"
 cookie=0x0, duration=345.729s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s2-eth1",dl_src=22:16:23:8b:9a:25,dl_dst=1a:f7:90:f3:d3:db actions=output:"s2-eth2"
 cookie=0x0, duration=345.718s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s2-eth3",dl_src=72:7d:a2:5f:00:d0,dl_dst=22:16:23:8b:9a:25 actions=output:"s2-eth1"
 cookie=0x0, duration=345.712s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s2-eth1",dl_src=22:16:23:8b:9a:25,dl_dst=72:7d:a2:5f:00:d0 actions=output:"s2-eth3"
 cookie=0x0, duration=345.695s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s2-eth2",dl_src=1a:f7:90:f3:d3:db,dl_dst=76:19:35:77:30:b7 actions=output:"s2-eth1"
 cookie=0x0, duration=345.685s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s2-eth1",dl_src=76:19:35:77:30:b7,dl_dst=1a:f7:90:f3:d3:db actions=output:"s2-eth2"
 cookie=0x0, duration=345.672s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s2-eth3",dl_src=72:7d:a2:5f:00:d0,dl_dst=76:19:35:77:30:b7 actions=output:"s2-eth1"
 cookie=0x0, duration=345.663s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s2-eth1",dl_src=76:19:35:77:30:b7,dl_dst=72:7d:a2:5f:00:d0 actions=output:"s2-eth3"
 cookie=0x0, duration=345.642s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s2-eth3",dl_src=72:7d:a2:5f:00:d0,dl_dst=1a:f7:90:f3:d3:db actions=output:"s2-eth2"
 cookie=0x0, duration=345.638s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s2-eth2",dl_src=1a:f7:90:f3:d3:db,dl_dst=72:7d:a2:5f:00:d0 actions=output:"s2-eth3"
 cookie=0x0, duration=369.600s, table=0, n_packets=70, n_bytes=5096, priority=0 actions=CONTROLLER:65535
```

- Diferença da fase 2 para a fase 3:

Na **Fase 2**, os switches estavam apenas conectados ao controlador, mas sem nenhuma lógica de encaminhamento implementada, todo pacote que chegava aos switches eram encaminhados ao controlador pelas mensagens Packet-In e como o controlador era vazio e não fazia nada com os pacotes eles eram descartados. Já na fase 3 o controlador não é mais vazio ele utiliza MAC Learning para aprender dinamicamente os endereços MAC e suas respectivas portas de entrada e envia os pacotes recebidos, além de aprender ele instala fluxos (regras) nos switches para que os próximos pacotes sejam enviados diretamente para o destino, sem ter que passar pelo controlador.

Fase 4: Políticas de Segurança e Controle (Firewall SDN)

Nessa ultima fase foi implementado o script “firewall_SDN.py”, que implementa um controlador que faz tudo que o da fase anterior fazia, mais a adição de um firewall que segue as seguintes regras estabelecidas: dropar qualquer pacote de ping (ICMP) que vá do h1 ao h4.

- Testes de ping:

  Pings executados para testar o firewall do controlador.

1. h1 ping h2 funcionando:

```
mininet> h1 ping -c 1 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.040 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.040/0.040/0.040/0.000 ms
```

2. h3 ping h4 funcionando:

```
mininet> h3 ping -c 1 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=5.25 ms

--- 10.0.0.4 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 5.254/5.254/5.254/0.000 ms
```

3. h1 ping h4 falha:

```
mininet> h1 ping -c 1 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.

--- 10.0.0.4 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms
```

- Teste de servidor:

Para testar que o firewall está bloqueando apenas pacotes ICMP do h1 para o h4, um servidor simples foi criado no h4 para o h1 fazer uma requisição a ele. A requisição foi concluída com sucesso, indicando que o firewall está funcionando como esperado.

```
mininet> h4 python3 -m http.server 80 &
mininet> h1 wget 10.0.0.4
--2026-05-01 01:26:52--  http://10.0.0.4/
Connecting to 10.0.0.4:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 596 [text/html]
Saving to: ‘index.html’

index.html          100%[===================>]     596  --.-KB/s    in 0s

2026-05-01 01:26:52 (44.4 MB/s) - ‘index.html’ saved [596/596]
```

- Regras:

Regras dos switches durante a fase 4, regra destacada é a regra de dropar pacotes ICMP do h1 para o h4.

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=363.310s, table=0, n_packets=1, n_bytes=98, priority=10,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.4 actions=drop
 cookie=0x0, duration=379.675s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth3",dl_src=8e:da:c8:a9:74:a3,dl_dst=06:0f:89:1f:1c:fb actions=output:"s1-eth2"
 cookie=0x0, duration=379.673s, table=0, n_packets=1, n_bytes=42, priority=1,in_port="s1-eth2",dl_src=06:0f:89:1f:1c:fb,dl_dst=8e:da:c8:a9:74:a3 actions=output:"s1-eth3"
 cookie=0x0, duration=363.313s, table=0, n_packets=1, n_bytes=42, priority=1,in_port="s1-eth1",dl_src=c6:1a:d1:96:1d:dd,dl_dst=06:0f:89:1f:1c:fb actions=output:"s1-eth2"
 cookie=0x0, duration=8.373s, table=0, n_packets=0, n_bytes=0, priority=1,in_port="s1-eth2",dl_src=06:0f:89:1f:1c:fb,dl_dst=c6:1a:d1:96:1d:dd actions=output:"s1-eth1"
 cookie=0x0, duration=417.225s, table=0, n_packets=58, n_bytes=4236, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2
 cookie=0x0, duration=375.502s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s2-eth3",dl_src=c6:1a:d1:96:1d:dd,dl_dst=66:4a:fa:98:81:a7 actions=output:"s2-eth2"
 cookie=0x0, duration=375.501s, table=0, n_packets=1, n_bytes=42, priority=1,in_port="s2-eth2",dl_src=66:4a:fa:98:81:a7,dl_dst=c6:1a:d1:96:1d:dd actions=output:"s2-eth3"
 cookie=0x0, duration=366.773s, table=0, n_packets=1, n_bytes=42, priority=1,in_port="s2-eth3",dl_src=c6:1a:d1:96:1d:dd,dl_dst=06:0f:89:1f:1c:fb actions=output:"s2-eth1"
 cookie=0x0, duration=11.827s, table=0, n_packets=0, n_bytes=0, priority=1,in_port="s2-eth1",dl_src=06:0f:89:1f:1c:fb,dl_dst=c6:1a:d1:96:1d:dd actions=output:"s2-eth3"
 cookie=0x0, duration=420.654s, table=0, n_packets=58, n_bytes=4228, priority=0 actions=CONTROLLER:65535
```